Primus Personal Assistant Project Documentation

Purpose features architecture opportunities and challenges

25 October 2026

Project overview

PRIMUS is a local-first personal assistant that helps a user interact with their computer through natural-language requests. It combines a local language model with tools for files, applications, browser navigation, reminders, document questions, image interpretation, and small coding projects. Users can reach it through a command line, desktop tray, keyboard shortcut, voice, or an authenticated local API. A local Mission Control dashboard manages coding workflows.

The central idea is to turn a request into a bounded sequence of tool actions and report what those tools actually confirmed. For example, a request to open Chrome can take a direct tool route, while creating a small program can involve planning, file creation, syntax checks, and review. The model interprets requests; Python tools perform the supported actions.

The problem it addresses

Everyday computer work often involves switching between applications, remembering paths, searching personal notes, and repeating simple steps. PRIMUS brings these activities into one assistant with local inference and configurable access to the user's folders. Voice and gestures provide additional ways to start tasks, although their recognition and supported commands remain limited.

Intended users and uses

- Individuals who want local assistance with routine desktop tasks and reminders.
- Students and researchers who want to ask questions about selected notes and documents.
- Developers who want help creating, checking, and running small Python or shell projects.
- Users exploring alternative input methods, subject to usability and accessibility testing.

Current project profile

Area	Current implementation
Main platform	Windows-focused deployment with some portable Python components
Main intelligence	Local Ollama model qwen3:4b-instruct-2507-q4_K_M
Other models	qwen3:1.7b for short answers and qwen3-vl:4b for vision
Interfaces	CLI, tray, Ctrl+Alt+P prompt, voice, localhost API, Mission Control
Storage	Local SQLite databases, output files, and redacted audit records
Approval policy	Automatic tool approval is enabled in the supplied configuration

PRIMUS is suitable for bounded personal workflows with review of consequential results. Broad feature coverage does not establish production readiness or universal task accuracy. The sections below describe implemented behavior and label proposed improvements separately. [S1–S3]

GitHub repository

https://github.com/raizel101/Primus-Personal-Assistant

Everyday assistant features

Natural language and direct shortcuts

The standard agent uses a plan–act–observe loop: it interprets the request, selects tools, examines their results, and continues within a configured limit. The current general-task limit is eight steps. Clear requests advertise a smaller relevant tool family to reduce prompt size; compound or unfamiliar requests retain a broader tool catalog.

Exact requests such as opening a supported app, listing outputs, listing reminders, or opening a named website can bypass model planning. These shortcuts still pass through authorization and auditing. A separate small-model route answers suitable short general-knowledge questions without tools or live information. If that model is unavailable, the service can fall back to the standard model. [S3]

Files and small code changes

PRIMUS can list folders, read UTF-8 text, create new text files, replace existing content, and make exact snippet patches. New files are read back for verification. Existing-file changes include version checks so a stale reviewed file is not silently changed. Python and JSON syntax can be checked without executing code.

Access is bounded by the project root and approved folders. The supplied configuration includes Downloads, Documents, Desktop, and one additional project folder. Credential-related locations and the assistant's protected runtime files are excluded by the bundled path guard. These boundaries apply to bundled tools; trusted Python plugins run with the process's operating-system permissions. [S2, S4]

Applications and windows

The fixed application catalog includes browsers, a text editor, calculator, file manager, and terminal when available. Windows Start app discovery adds name-based searches and exact launches for other registered apps. Opening an application does not grant access to its private data or account.

Windows tools can list, focus, minimize, restore, maximize, arrange, or request closure of windows. Accessibility tools inspect exposed controls and can activate a control or replace text in a non-password edit field. General desktop input supports limited keyboard and mouse actions. Some tools verify window state or focus; delivery of a click or text still does not prove that an application completed a save, send, or transaction. [S4]

Browser assistance

PRIMUS controls installed Chrome through Playwright using a separate profile. It can navigate HTTPS pages, read visible page text, click a visible CSS-selected element, and fill an eligible field. Password and file inputs are blocked, and downloads are disabled in this automation browser. It does not inherit the user's normal browser cookies or saved passwords.

Opening a website requires internet access. The browser is enabled independently of the generic search and Google connectors, which are disabled in the supplied configuration. The approval behavior for actions depends on the policy described later in this document. [S1, S2, S4]

Memory document knowledge and scheduling

Three kinds of context

PRIMUS separates temporary conversation context, structured recent-task context, and saved facts or preferences. They serve different purposes and have different persistence behavior.

Context type	What it contains	Lifetime
Recent conversation	A bounded set of recent user and assistant messages	Process memory
Recent task references	Last successful app, file, folder, target, or reminder reference	SQLite with 30-day retention configured
Facts and preferences	Explicitly requested saved information	SQLite until edited or removed

Recent task references help resolve requests such as “open it again” without storing a complete transcript. Persistent fact or preference changes use the permission gate. In automatic mode, policy approval satisfies that gate without a separate human prompt. The assistant is not designed to silently learn preferences from every conversation. [S2, S5]

Questions about selected documents

The knowledge system implements retrieval-augmented generation, or RAG. It first searches indexed passages and then supplies relevant text to the local language model. SQLite FTS5 with BM25 ranking provides keyword-based retrieval; the system does not retrain the model or use dense semantic embeddings.

Supported sources are UTF-8 TXT, Markdown, and RST files, plus text PDFs when the selected Python runtime has pypdf. Current limits include 1 MB per source, 50 pages per PDF, 200 chunks per source, and 100 indexed documents. Scanned documents require an additional OCR capability that is not implemented here.

Indexing persists document text locally. Search checks the approved source version; changed or unavailable sources need attention before they can support a current answer. The explicit `knowledge:` or `rag:` route retrieves before generation. Answers include checked citation labels and source information. A valid citation label establishes provenance, while the user must still assess whether the passage supports the interpretation. Forgetting removes the index entry and preserves the original file. [S6]

Reminders and recurring tasks

The scheduler stores reminder and agent-task definitions in SQLite, supports recurrence, and checks due work while its process is running. It is not an independent cloud scheduler. Sleeping, stopping the service, or restarting during an action can affect delivery; an interrupted job may run again if completion was not recorded. Under manual policy, unattended agent jobs cannot approve actions requiring interactive consent. Automatic policy permits supported actions through the same gate. [S5]

Optional external integrations

Read-only Google Gmail and Calendar connectors and Brave search exist but require credentials and explicit configuration. They are disabled in this snapshot. Cloud model providers remain disabled. These optional features should be described as integration paths rather than ready-to-use account connections. [S1, S2]

Voice vision and gesture interaction

Local voice assistant

Voice mode listens for a supported direct address, transcribes the command locally with faster-whisper, submits it to the local task service, and speaks the result with Piper. The configured speech model is faster-whisper-base.en, and the voice is en_US-lessac-low. Automatic speech-device selection attempts compatible NVIDIA acceleration and falls back to CPU when its startup probe fails.

Wake acceptance uses two unprompted recognition passes. A bounded capture worker keeps recent utterances in memory while wake checks run. Microphone capture closes before spoken output, so the interaction is turn-based and does not support interrupting PRIMUS while it speaks. Noise, unclear pronunciation, and pauses can affect recognition. A documented matcher also accepts “Premus” after Hey, OK, or Okay; that narrow alias does not imply general fuzzy wake matching.

The optional animated orb displays accepted-wake, processing, speaking, and error states. It is a state indicator rather than a live audio waveform. Tray startup is configured to start voice automatically; Windows sign-in startup additionally depends on the separate always-on task being installed. Audio is held in memory rather than saved as recordings by the voice workflow. [S1, S2, S7]

Image and window understanding

PRIMUS can interpret a selected JPEG, PNG, or WebP in an approved location. Scene, detail, and text modes use a separate local vision model. The user can select a focus region to improve attention to a small object or text. Input limits include 10 MB and 25 million pixels; the image is oriented, sanitized, and resized in memory.

On Windows, the assistant can also capture a specific visible application window in memory and describe it through the local vision server. Recognized credential windows and password-related fields are excluded. Vision can misread text or miss details, and a description does not establish that an application action succeeded. The private vision server requires separate setup and availability. [S7]

Camera gestures

The gesture preview uses local ONNX hand models through OpenCV on CPU. It recognizes a limited set of static poses, including open palm, closed fist, pointing, victory, thumb up or down, and pinch. Stable holds, hand release, cooldowns, and one pending request at a time reduce repeated triggers.

Gesture commands are off for each new camera session until the user enables them. The implemented shortcuts use thumb-up for YouTube, victory for Chrome, and pointing for Google. A gesture starts a request; it is not itself approval. With automatic approval enabled, the configured policy can authorize the resulting tool action without a dialog. Frames and landmarks remain in memory. This is limited pose recognition, and it does not translate sign language. [S7]

Coding workflows teams and formal planning

Mission Control

Mission Control is a local dashboard for small coding tasks. Its planner, coder, and verifier use separate model conversations and run sequentially. One coding mission runs at a time to limit local resource pressure.

1. The planner produces a validated plan and acceptance criteria.
2. The coder creates or changes artifacts inside a fresh mission workspace.
3. The verifier re-reads actual files, records hashes, performs supported parsing checks, and reviews the result against the goal.
4. The workflow can request a bounded correction when review fails.
5. A requested Python or shell run uses the offline sandbox and records execution evidence.

The dashboard shows role states, tool events, file previews, review results, and execution records. Mission limits include 12 files of 100 KB each and 24 KB of text for model review. Python and JSON checks establish syntax; HTML parsing is not a browser layout test. Other text formats receive model review. A workflow marked complete still needs task-appropriate acceptance checks. [S8]

Sandboxed execution and runtime repair

On this Windows deployment, supported scripts run in Ubuntu WSL with Bubblewrap. The sandbox restricts network, mounted inputs, writable output, runtime, and resources. It executes Python or POSIX shell, rather than arbitrary Windows executable or PowerShell files. Current limits include 25 seconds, 1 GB of address space, 10 MB per file, and 50 MB of output.

Mission runs can snapshot a bounded multi-file project, pin its files by hash, and reject changed inputs before launch. A nonzero program exit can trigger one focused runtime-repair pass, followed by fresh verification and a separately authorized rerun. Independent test files are protected from that repair pass. A zero exit with wrong output is not detected by the nonzero-exit trigger. Authorization follows the current manual or automatic policy. [S8]

Parallel inspection teams

The team feature divides suitable requests into independent local read-only subtasks. The current configuration allows two concurrent workers and up to three subtasks. Workers have separate conversations and return separate results. Network, writes, application control, and dynamically tiered tools are excluded. Local model scheduling may serialize inference, so parallel execution does not guarantee lower latency. [S9]

Symbolic planning

The formal planner accepts a supplied JSON STRIPS model containing facts, preconditions, and add/delete effects. Breadth-first search finds a shortest unit-cost plan within bounded limits, and an independent replay checks each transition and final goal. It uses no language model.

The current limits are 32 facts, 64 actions, 4,096 visited states, depth 16, and two seconds of search. Full PDDL parsing, automatic grounding, temporal planning, and direct tool execution are excluded. A valid symbolic plan proves only that its declared model reaches its goal; it does not execute laptop actions or verify real-world success. [S10]

Architecture and current configuration

How a request moves through the system

A CLI, tray, voice, gesture, or API client submits a task to the shared service. Explicit knowledge and formal-planning requests take specialized routes. Suitable simple questions use the fast-answer model, exact shortcuts use direct plugins, and other requests use the standard agent. Plugin inputs are validated, the permission policy is applied, and tool results are audited. Successful tools may update structured context; task history records status and hashes. [S3]

Main project components

Location	Responsibility
primus.py and src/primus/cli.py	Project entry point and command-line commands
src/primus/service.py and agent.py	Request routing and bounded model/tool execution
src/primus/plugins.py and plugins/	Plugin contract, discovery, schemas, and abilities
src/primus/security.py and audit.py	Path restrictions, authorization, and audit redaction
src/primus/storage.py and scheduler.py	Facts, task context, history, and scheduled work
knowledge.py and mcp_client.py	Document retrieval and configured tool connections
missions.py and mission_web/	Coding orchestration and local dashboard
voice.py, vision.py, and gestures.py	Speech, window vision, and hand-pose processing
tests/, scripts/, and examples/	Automated checks, setup utilities, and demonstrations
data/, logs/, models/, and outputs/	Local state, records, models, and generated artifacts

The source filenames in this table are relative to `src/primus/` unless a different location is shown.

Selected configuration values

Setting	Supplied value and meaning
model.endpoint	127.0.0.1:11434 for the main Ollama chat model
fast_answer and vision endpoints	127.0.0.1:11435 for the private local model server
local_api	127.0.0.1:8765 for authenticated local clients
local_mcp	Enabled on 127.0.0.1:8767
permissions.auto_approve	true for automatic authorization of tool tiers
browser and installed_apps	Enabled
knowledge, missions, vision, gestures	Enabled; each still needs its runtime prerequisites
network_plugins_enabled and google.enabled	false

The MCP client connects only to explicitly configured localhost HTTP servers. The supplied first-party server exposes the limited `primus_status` tool. MCP is a tool connection protocol, not an extra model. It does not automatically make all external services available. Third-party servers run outside the script sandbox and require their own trust assessment. [S2, S6]

Security privacy and user control

Current automatic approval policy

The supplied `config.json` sets `permissions.auto_approve=true`. The central authorization function therefore approves every recognized tool tier, including overwrites, deletion-class operations, browser actions, MCP calls, and sandbox execution, without an individual prompt. It records these as automatic policy approvals rather than individual human decisions.

Earlier setup guides describe exact-action dialogs. Those descriptions apply when automatic mode is off. To restore that mode, set the flag to false and restart PRIMUS; the configured per-tier policies then apply. This document does not change the policy. The authenticated health response reports the policy loaded by the running service, which may differ from an edited configuration until restart. [S2, S4]

Controls that still apply

- Approved-folder and credential-path restrictions in the bundled file tools.
- Protected assistant source, configuration, audit, and runtime paths.
- Tool-schema validation, action-specific checks, and file-version checks.
- Disabled cloud and optional network integrations.
- Script sandbox resource, filesystem, and network boundaries.
- Task cancellation, bounded step counts, and the global STOP switch.

Automatic approval changes authorization behavior; it does not grant administrator rights, expand folder access, or start unrequested workflows. The kill switch prevents new work and is checked during some long-running operations, but it cannot reverse completed actions or guarantee interruption of an operating-system call already in flight. [S4, S8]

What remains on disk

Data	Persistence and implication
Facts, preferences, schedules	Content stored locally in primus.sqlite3
Recent task context	Short references stored locally with expiry
Document knowledge	Indexed passages stored in knowledge.sqlite3
Task history	Status, lineage, timestamps, and request/result hashes
Tool audits	Metadata with sensitive fields redacted or hashed
Generated files	Retained in output workspaces
API and connector credentials	OS keychain rather than ordinary configuration text

Voice recordings, camera frames, landmarks, and window screenshots are not retained by the described workflows. That does not mean the assistant stores no personal information: indexed documents, saved memories, paths, schedules, and outputs can still be sensitive. Audit redaction is selective, and local storage should be protected through the operating-system account and appropriate retention practices. [S5–S7]

The localhost API uses authentication and surface checks. Mission Control uses a short-lived launch exchange and scoped session cookie. These measures do not isolate PRIMUS from malicious software already running as the same operating-system user. Local inference supports privacy, while trusted plugins and MCP servers remain separate trust boundaries. [S1, S6, S8]

Opportunities for further development

The following opportunities are recommendations derived from the current implementation. They are proposed work, rather than claims that these capabilities already exist or that demand has been measured.

Improve the daily assistant experience

The strongest near-term opportunity is making common requests predictable: faster wake recognition, clear progress and error messages, fewer ambiguous app targets, and consistent reporting of verified results. A focused collection of frequently used tasks would make it easier to tune the small local model and detect regressions.

A dedicated local wake detector, more deliberate audio-device selection, and latency tracing could improve voice interaction. Broader language support and barge-in would extend usefulness, but require separate model, hardware, and physical usability testing. [S1, S7]

Develop personal knowledge support

Document search could expand to OCR, richer formats, semantic retrieval, and controlled folder update detection. These additions should preserve explicit source selection, version tracking, and visible removal controls. Evaluation should test whether answers are supported by the retrieved text, beyond checking citation-label validity. [S6]

Strengthen task verification

Application-specific connectors could verify saves and edits more reliably than generic clicks. Coding missions could add richer project checks and representative acceptance tests. Persistent mission details and deliberate restart recovery could make longer tasks easier to resume without repeating side effects. [S4, S8]

Prioritize opportunities by outcome

Priority	Proposed improvement	Useful success measure
High	Visible policy state and narrower per-tool approval controls	Users can explain which actions run automatically
High	Voice reliability and latency instrumentation	Wake misses, false accepts, and end-to-end response time
High	Reproducible installation and diagnostics	A fresh machine completes the documented smoke checks
High	Better outcome verification for frequent workflows	Measured task completion with incorrect-success reports tracked
Medium	OCR, semantic retrieval, and answer-support evaluation	Retrieval recall and supported-answer rate on reviewed questions
Medium	Durable mission recovery and schedule deduplication	Restart tests avoid lost work and duplicate side effects
Medium	Reviewed MCP and application connectors	Connector-specific success and failure evidence
Later	More OS adapters, multilingual voice, and mobile clients	Passing platform and usability acceptance checks

These are suggested measures; this document does not assign current baseline scores or delivery dates. A personal productivity pilot, a document-study assistant, or an accessibility exploration would each need its own small set of representative tasks before broader positioning. Existing plugin contracts and local APIs offer extension points, while testing should determine which use cases are dependable. [S3, S11]

Challenges limitations and development priorities

Main engineering challenges

Challenge	Why it matters	Recommended response
Local compute and model contention	Speech, chat, and vision compete for memory and inference time	Profile cold/warm latency and tune model residency and concurrency
Recognition and model errors	Wake phrases, intent selection, code, and visual descriptions can be wrong	Maintain representative tasks and physical device checks
Fragile application automation	Focus, selectors, controls, and layout can change	Prefer semantic controls or app APIs and verify final state
Broad automatic authorization	A mistaken tool choice can have immediate effects	Make policy visible and support narrower authorization scopes
Retrieval gaps	Keyword matching misses synonyms; citations can support a wrong interpretation	Test retrieval and answer support separately
Service restarts	Mission details may be lost and interrupted jobs may repeat	Add deliberate recovery and idempotent job handling
Platform differences	Window, accessibility, and vision adapters are Windows-specific	Publish a feature matrix and fail closed on unsupported backends
Plugin and MCP trust	Extensions may run beyond bundled tool or script boundaries	Review code and permissions before enabling an extension

Important scope limits

The fast-answer route has no tools or current data. General browser and desktop actions do not establish transaction success. The current knowledge index has no OCR or semantic embedding retrieval. Formal planning does not execute tools or prove the real machine state. Parallel teams are restricted to local read-only work. The voice system is turn-based, and gestures cover a small static-pose vocabulary. [S1, S6–S10]

Suggested development sequence

1. Stabilize the current installation, policy visibility, and error reporting. Record a small set of repeatable end-to-end tasks and establish baselines.
2. Improve voice recognition, task latency, retrieval quality, and final-state verification against those tasks.
3. Add durable mission recovery and scheduler safeguards. Test restarts, cancellations, changed files, and repeated requests.
4. Expand integrations and platforms only after defining adapter-specific success checks and data-access boundaries.

The repository includes tests for core phases, permissions, voice capture, browser recovery, coding missions, MCP, knowledge, formal planning, and other subsystems. README entries contain historical pass counts and selected live demonstrations from different development increments. They should not be combined into a current certification. A release should record the exact source revision, environment, full-suite result, and relevant physical or live checks. [S1, S12]

Setup usage and maintenance

Prerequisites

The base application requires Python 3.11 or newer, a local Ollama service, and the configured main model. Setup does not automatically install Ollama or pull the main model. Optional packages and model files support the tray, keychain, browser, speech, vision, and gestures. Sandboxed script execution on Windows additionally requires the configured Ubuntu WSL environment and Bubblewrap. Feature enablement alone does not establish that these dependencies are ready. [S1]

Start from the project folder

The following PowerShell commands assume `python` resolves to the compatible interpreter used for PRIMUS. If it does not, substitute that interpreter's absolute path. Run the diagnostic commands before starting interactive features.

```
Set-Location -LiteralPath 'C:\Users\Subhajeet Bose\Downloads\Primus'
python primus.py doctor
python primus.py plugins
python primus.py voice-doctor
python primus.py gesture-doctor
```

Common commands

```
python primus.py ask "list my outputs folder"
python primus.py ask "open Google Chrome"
python primus.py ask "open YouTube"
python primus.py ask "Quick: explain recursion"
python primus.py knowledge index "outputs/notes.md"
python primus.py knowledge ask "What do my notes say?"
python primus.py schedule-list
python primus.py context-list
python primus.py dashboard
python primus.py tray
```

Use `voice` for a foreground voice session, `gestures` for the camera preview, and `serve` for the authenticated API and scheduler. Avoid competing service instances on the same ports. The tray already starts voice when configured to do so. Use `stop` to activate the global kill switch and `resume` when ready to clear it. These commands perform real actions under the currently loaded approval policy. [S1, S3]

Operational maintenance

Keep the project folder and its configured root together. Review approved folders and enabled integrations when moving the installation. Restart PRIMUS after changing configuration; use health information to confirm the loaded approval mode. The private vision/fast-model server needs its documented restart procedure after a reboot. Back up desired outputs and local databases with an approach appropriate for SQLite consistency, and protect copies containing indexed text or saved facts.

Run the repository test command when validating a code release: `python -m unittest discover -s tests -v`. Use the project's temp directory as described in the README. Follow with physical microphone/camera checks and relevant live app tests, since fixtures alone cannot verify device behavior. Detailed setup and extension instructions remain in the source guides listed on the next page. [S1, S11, S12]

Project source guide

Project root: `C:\Users\Subhajeet Bose\Downloads\Primus`

References below identify the local documentation, configuration, and implementation supporting this project description. Paths are relative to the project root. Configuration values describe the supplied file on 25 October 2026; running-process state should be checked separately. Opportunities and development priorities are recommendations.

Reference	Source files and subject
S1	README.md — overview, setup, commands, limitations, feature increments, and current approval-mode addendum
S2	config.json; src/primus/config.py — selected models, endpoints, feature flags, approved folders, and policy validation
S3	primus.py; src/primus/cli.py, service.py, agent.py, fast_intents.py, fast_answer.py, tool_router.py — interfaces and request routing
S4	src/primus/security.py; plugins/files.py, coding.py, desktop.py, installed_apps.py, windows.py, accessibility.py, browser.py — tool boundaries and desktop behavior
S5	src/primus/storage.py, context_memory.py, scheduler.py, audit.py; plugins/memory.py, schedule.py — persistence, scheduling, and redaction
S6	MCP_RAG_GUIDE.md; src/primus/knowledge.py, mcp_client.py, mcp_server.py; plugins/knowledge.py, mcp.py — document retrieval and local tool connections
S7	README.md voice, vision, and gesture sections; src/primus/voice.py, vision.py, gestures.py, gesture_actions.py, image_service.py — multimodal interaction
S8	README.md Mission Control section; src/primus/missions.py, script_runner.py, api.py, dashboard.py, mission_web/ — coding workflows, execution, and dashboard access
S9	README.md Parallel task agents section; src/primus/team.py — read-only worker coordination
S10	FORMAL_PLANNING_GUIDE.md; src/primus/planning.py; plugins/planning.py — bounded symbolic planning and replay
S11	PLUGIN_GUIDE.md; src/primus/plugins.py; requirements-phase2.txt, requirements-phase3.txt, requirements-browser.txt, requirements-gestures.txt — extension contract and optional dependencies
S12	tests/; scripts/demo-formal-planning.py, test-syllabus-rag.py, test-runtime-repair-live.py, test-website-opening-live.py — automated coverage and demonstration entry points

Terms used in this document

Ollama serves the local language and vision models. A plugin is a trusted Python ability with a declared input schema and permission tier. RAG retrieves document passages before answer generation. MCP connects the assistant to explicitly configured tool servers. STRIPS represents planning actions as preconditions and changes to symbolic facts. An audit hash identifies recorded content or a reviewed version; it does not establish that the content is true or that an intended real-world outcome occurred.